📄 Project - High Level Design on Telecom DevOps Messaging Platform

Course Name: DevOps & Cloud Engineering

Institution Name: Medicaps University – Datagami Skill Based Course

Sr. no	Student Name	Enrolment Number
1.	Samiksha Saxena	EN22CS301853
2.	Sanand Alshi	EN22CS301859
3.	Sanidhya Somani	EN22CS301865
4.	Sanidhya Yadav	EN22CS301866
5.	Sarthak Chajjed	EN22CS301874

1. Introduction

1.1 Scope of the document

This document provides a High-Level Design (HLD) for the Telecom Messaging Platform built using DevOps practices. It outlines system architecture, components, workflows, and deployment strategy using containerization and Kubernetes.

1.2 Intended Audience

Developers

DevOps Engineers

QA/Test Engineers

Project Evaluators

System Architects

1.3 System Overview

The Telecom Messaging Platform is a simulated API-based system that mimics real-world telecom services such as message delivery, user management, and balance tracking.

The system is built using:

- Backend APIs (Flask)
- Containerization (Docker)
- CI/CD (Jenkins)
- Orchestration (Kubernetes)

It provides a web-based UI for interaction and supports API testing via tools like Postman.

2. System Design

2.1 Application Design

The application follows a client-server architecture:

Frontend: HTML/CSS/JS dashboard

Backend: Flask REST API

Containerization: Docker

Orchestration: Kubernetes

2.2 Process Flow

User → UI → API Request → Flask Backend → Process Logic → Response → UI Display

CI/CD Flow:

Code → Jenkins → Docker Build → DockerHub → Kubernetes Deployment

2.3 Information Flow

- User inputs data via UI
- UI sends HTTP request to backend
- Backend processes request
- Response returned in JSON
- UI displays response

2.4 Components Design

1. Frontend Layer

- HTML dashboard

- Interactive UI for API calls

2. Backend Layer

- Flask-based REST API
- Handles business logic

3. Container Layer

- Docker container encapsulates application

4. CI/CD Layer

- Jenkins automates build and deployment pipeline

5. Orchestration Layer

- Kubernetes manages containers and scaling

2.5 Key Design Considerations

- Scalability using Kubernetes
- Portability using Docker
- Automation using Jenkins
- Modular API design
- Lightweight frontend for quick interaction

2.6 API Catalogue

Endpoint	Method	Description
/health	GET	Health check for Kubernetes
/status	GET	Service status
/send-message	POST	Send SMS
/check-balance	GET	Check balance
/create-user	POST	Create user
/users	GET	List users
/delete-user/{id}	DELETE	Delete user
/recharge	POST	Recharge balance
/message-status/{id}	GET	Message status
/delivery-report	GET	Delivery statistics
/service-info	GET	Service metadata

2.7 Architecture Diagram

2.7.1 High-Level Architecture Diagram

```
+----------------------+
|      End User        |
| (Browser / Postman)  |
+----------+-----------+
           |
           v
+----------------------+
|     Frontend UI      |
|   (HTML/CSS/JS)      |
+----------+-----------+
           |
           v
+----------------------+
|   Flask Backend API  |
|   (Python Server)    |
+----------+-----------+
           |
           v
+----------------------+
|   Docker Container   |
| (App Packaging Unit) |
+----------+-----------+
           |
           v
+----------------------+
|    Kubernetes        |
| (Deployment + Svc)   |
+----------+-----------+
           |
           v
+----------------------+
```

```
| Cloud / Server |
| (AWS / Local VM) |
+----------+
```

2.7.2 CI/CD Pipeline Architecture

```
+----------+
| Source Code |
| (Git Repository) |
+---------+---------+
|
v
+----------+
| Jenkins |
| (CI/CD Server) |
+---------+---------+
|
Build Docker Image
|
v
+----------+
| DockerHub |
| (Image Registry) |
+---------+---------+
|
Pull Image
|
v
+----------+
| Kubernetes |
| Deployment |
+----------+
```

2.7.3 Kubernetes Architecture

```
+-----------------------+
|    Kubernetes Node    |
+-----------------------+
            |
+----------+----------+
|                     |
v                     v
+---------------+     +---------------+
|   Pod         |     |   Pod         |
| (Flask App)   |     | (Flask App)   |
+-------+-------+     +-------+-------+
|                     |
+----------+----------+
            |
            v
+-------------------+
|   Service Layer   |
| (NodePort / LB)   |
+-------------------+
            |
            v
External Access
```

3. Data Design

3.1 Data Model

- Users stored in in-memory list
- Balance stored as variable
- No persistent database used

3.2 Data Access Mechanism

- REST APIs using JSON
- HTTP methods: GET, POST, DELETE

3.3 Data Retention Policies

- Data is temporary (in-memory)
- Lost on container restart

3.4 Data Migration

Not applicable (no database used)

4. Interfaces

- Web UI (HTML dashboard)
- REST APIs
- Postman for testing

5. State and Session Management

- Stateless architecture
- Each request handled independently
- No session storage

6. Caching

- No caching implemented
- Real-time responses used

7. Non-Functional Requirements

7.1 Security Aspects

- API-based communication
- Input validation
- Container isolation using Docker

7.2 Performance Aspects

- Lightweight Flask backend
- Fast API response
- Kubernetes ensures scalability
- Load handling via container replication

8. References

- Docker Documentation
- Kubernetes Documentation
- Jenkins Documentation
- Flask Framework Docs